

BUILD MANUAL

RaceChrono DIY CAN-Bus アダプタ 製作手順書

ESP32 + SN65HVD230 / Abarth 595 シリーズ4 向け

対象車両	Abarth 595 シリーズ4 (Fiat 500 / 312系)
マイコン	Freenove ESP32-WROOM ボード (FNK0090B)
ファームウェア	kevenduchesneau / racechrono-diy-esp32 (上流コミット c2290a9)
動作モード	受信専用 (Listen Only / TX 無接続)
版数	Rev. 1.6 (既知 CAN ID 候補の追加、電源スイッチ・外付けコンデンサの削除)
作成日	2026年9月5日
ビルド確認	arduino-esp32 3.3.11 / NimBLE-Arduino 1.4.3 / EasyLogger 1.1.4 / ESP32 Dev Module

1. 本書の概要と設計方針

本書は、Abarth 595 シリーズ4の OBD-II ポートから CAN バスのデータを読み取り、Bluetooth LE 経由でスマートフォンアプリ「RaceChrono Pro」へ送信する DIY データロガーの製作手順書です。Rev.1.0 の技術レビューと、その後の設計簡素化（保護部品・抵抗・電源スイッチ・外付けコンデンサの削除、TXD 開放）、Fiat 500 系の既知 CAN ID の反映を含みます。変更の経緯は末尾の改訂履歴を参照してください。

設計上の4つの原則

原則	内容
完全受信専用	車両の CAN バスへ一切の信号を送出しない。ファームウェアの Listen Only モードに加え、 トランシーバの送信入力 D を無接続（開放＝レセッシブ固定） にし、ESP32 側から物理的に駆動できないようにする二重の対策を採る。
バス負荷ゼロ	トランシーバ基板上的の終端抵抗を除去し、車両側バスのインピーダンスに影響を与えない。
電源の切り離し	OBD-II の 12V は常時通電のため、使わないときは装置を OBD ポートから物理的に抜く。電源スイッチは設けない。
段階的に確認	机上（USB 給電）→ 机上（12V 給電）→ 車両（USB 給電・12V 線は未接続）→ 車両（12V 運用）の順に、1 段階ずつ動作を確認してから次へ進む。

重要：車両側の安全に関する前提

本装置は走行中の車両の通信バスに接続します。誤配線や設定ミスはパワーステアリング・ABS・エンジン制御に影響を及ぼす可能性があります。**第9章のベンチテストを完了するまで、車両に接続しないでください。**また、公道での初回接続は避け、必ず安全な場所での停車状態から確認を始めてください。

2. 部品表（BOM）

Rev.1.0 から削除した部品：ヒューズ、逆接続保護ダイオード、TVS、10k Ω 抵抗、電源スイッチ、電解コンデンサ×2（経緯は改訂履歴）。残りは 7 品目。

部品	仕様・型番	役割と備考
マイクロコントローラ	Freenove ESP32-WROOM 開発ボード（FNK0090B）	メイン処理・BLE通信。ESP32-WROOM-32E 搭載、デュアルコア240MHz / 4MBフラッシュ / BT 4.2 LE、USB-シリアルは CH340。ピンヘッダは実装済みのため、薄型化のため除去を推奨（第4章）。
CANトランシーバ	SN65HVD230 ブレイクアウト基板	CAN差動信号を3.3Vロジックへ変換。 基板上の120Ω終端抵抗は必ず除去すること （第4章）。
OBD-IIコネクタ	SAE J1962 Type A 16ピン オス（金属ラッチ・圧着端子）	車両ポートへの接続。金属ラッチにより強いGでの脱落を防止。付属オープンバレル端子は AWG24～20 対応。
降圧モジュール	MP1584EN 可変DC-DCモジュール	車両12Vを5Vへ降圧。 接続前に必ず5.0Vへ調整 （第6章）。
配線材	AWG24 耐熱シリコンワイヤー	モジュール間結線。IWISS SN-QC-WC で圧着可能、狭い筐体でも取り回し良好。
絶縁・固定材	カプトンテープ、熱収縮チューブ、結束バンド	ショート防止と、筐体・車両ハーネスの物理固定。
筐体	3Dプリント品（タイラップ用スリット付き）	OBDポート周辺は膝が当たるため、可能な限り薄型に設計する。夏季の車内温度を考慮し PLA は避ける。

補足：予備のボードについて

Freenove のボードは2個パックで購入済みのため、1枚はベンチテスト・練習用として温存することを推奨します。ピンヘッダ除去やはんだ付けの失敗リスクに対する保険になります。

3. 対象車両のCANバス仕様

Abarth 595 シリーズ4は Fiat 500（312系）プラットフォームを共有しており、**高速側と低速側の2系統のCANバス**を持ちます。Fiat 500（312）の C-CAN では、常時放送されるフレームも **29 ビット（拡張）ID** で観測されています（例：0x0218A006 車輪速。2010 年式 500 1.2 の公開ログ）。Abarth 595 も同じ形式である可能性が高い（有力）ため、本書の手順は 29 ビット ID を前提に書いています。ファームウェアは全 ID 受け入れなので設定変更は不要です。

バス	速度	OBDコネクタ	主に流れるデータ
CAN-C（高速）	500 kbps	ピン 6 / 14	エンジン制御、ABS/ESC、変速機系。RPM・車速・アクセル開度・ブレーキ圧など
CAN-B（低速）	50 kbps	ピン 1 / 9	ボディ系、メーター、電動パワーステアリング、快適装備。本書では使用しない

本手順書は**ピン6/14の CAN-C（500kbps）**を対象とします。ファームウェアの初期設定が 500kbps のため、ビットレートの変更は不要です（ピン割当は AlfaOBD の Fiat/Alfa 向け資料と一致）。

舵角は CAN-C 側にある見込み（有力）

AiM の「FIAT_ABARTH – CAN STANDARD」プロトコル（Fiat 500 Abarth 2008 年以降）と Racelogic の「Fiat 500 2007-」CAN データベースは、どちらも OBD ピン 6/14（500kbps）を聞くだけで舵角・ブレーキ圧・4 輪速・アクセル開度・回転数などを取得しています。したがって舵角のために CAN-B（ピン 1/9）用の 2 台目を作る必要はない見込みです。CAN-C で見つからなかった場合に限り、2 台目（B-CAN はフォールトトレラント方式の可能性が高く SN65HVD230 での受信は要確認、RaceChrono の 2 台同時接続も要確認）を検討します。

整備済みの CAN ID 一覧は無いが、手がかりは多い

MX-5 や GT86 のように RaceChrono 用に整備された一覧は Abarth 595 向けには公開されていません。ただし Fiat 500 (312) 系は同じ CAN マトリクスを共有しており、公開情報から次の候補が得られます。第10章では**この候補を最初に確認**し、足りないものだけスニファで探します。

CAN ID (29bit)	内容	出典・確度
0x0218A006	4 輪速。8 バイトに 16 ビット × 4 (FL/FR/RL/RR の順は要確認)。停車時は各 0x002C	Fiat 500 1.2 (2010) の公開ログ (P1kachu)。有力
0x0210A006	車速。バイト 4-5 (ビッグエンディアン) ÷ 128 = km/h	Fiat 500L (FiatMon) で復号、Fiat 500 (2010) でも同 ID が車速に連動。有力
0x0618A001	エンジン回転数。バイト 2-3 (ビッグエンディアン) = rpm	Fiat 500L (FiatMon)。Fiat 500 (2010) にも同 ID が存在。五分
0x0810A000	ブレーキ。バイト 2 の上位ニブルが踏力で増加 (0x1x 離し / 0x3x 軽踏み / 0x7x 強踏み)	Fiat 500 (2010) の公開ログ。五分 (圧力のスケールは未確認)
0x0A18A000	状態ビット。バイト 0 ビット 5 = パーキングブレーキ、バイト 2 = ドア / キー状態	Fiat 500 (2010) の公開ログ。五分
0x0628A001	バイト 5 にクラッチ状態ビット	Fiat 500 (2010) の公開ログ。五分
0x0018A001	水温。バイト 3 - 40 = °C	Fiat 500L (FiatMon)。五分
(未特定)	舵角・ブレーキ圧・アクセル開度・ヨーレート・横G	AiM / Racelogic は取得済みだが ID 非公開。スニファで特定する (第10章)

出典：github.com/P1kachu/talking-with-cars (notes/fiat-500.txt)、github.com/VeryBusyBee/FiatMon (Fiat 500L)、Racelogic Vehicle CAN Database 「Fiat 500 2007-」 (チャンネル一覧のみ公開。ID は .REF ファイル内、可読かは要確認)、AiM InfoTech 「Fiat 500 Abarth (from 2008 onward)」。CANchecked 社は 595 向け製品で読取り済みだが非公開。

4. 事前加工 (組み立て前に必ず実施)

4-1. SN65HVD230 基板の120Ω終端抵抗を除去する

市販の SN65HVD230 ブレイクアウト基板の多くには、CANH-CANL 間に 120Ω の終端抵抗が実装されています。車両の CAN バスは既に両端が 120Ω で終端されているため、これを追加すると3本並列となりバスインピーダンスが約 40Ω まで低下し、通信エラーの原因となります。

- 基板上で CANH と CANL の端子間に配置された抵抗 (表面実装、刻印「121」または「1200」) を特定する。
- はんだごてで両端を温めながらピンセットで除去する。ジャンパ形式の基板であればカットする。
- テスターで CANH-CANL 間の抵抗値を測定し、120Ω 付近でないことを確認する。** トランシーバ内部の入力抵抗により数十 kΩ 程度が読める、または OL 表示になる。どちらも正常。

4-2. Rs (スロープ制御) ピンの確認

Rs ピンがジャンパで切り替え可能な基板の場合は、高速モード (Rs をグランド側) に設定します。抵抗で固定されている基板はそのまま問題ありません。Rs は主に送信側のスルーレートを決めるピンで、GND 側 (高速) または抵抗で GND に落とした状態 (スロープ制御) であれば受信には影響しません。

4-3. ESP32 ボードのピンヘッダ除去（任意）

Freenove のボードはピンヘッダがはんだ付け済みで出荷されます（2列 × 20 = 40ピン）。一括で吸い取るとランドが剥がれるため、以下の順序で作業してください。

1. ピンヘッダの黒い樹脂部分をニッパーで破壊し、除去する。
2. 残った金属ピンを1本ずつラジオペンチで掴み、はんだごてで温めながら引き抜く。
3. すべて抜き終わったら、はんだ吸取り線でスルーホールを整える。

※ 薄型化が不要であれば、この工程は省略しても機能上の問題はありません。

5. 結線図

5-1. 電源系統

接続元	経由	接続先
OBD-II ピン16（+12V 常時）	—	MP1584EN IN+
OBD-II ピン4 および 5（GND）	—	MP1584EN IN- / ESP32 GND / SN65HVD230 GND
MP1584EN OUT+（5.0V調整済）	—	ESP32 5V ピン（Freenove ボードの表記 は「5V」）

※ 外付けのコンデンサは付けない（MP1584EN モジュールの入出力コンデンサと ESP32 ボード側のコンデンサで足りる見込み・有力）。BLE 接続中に ESP32 がリセットを繰り返す場合だけ、ESP32 の 5V / GND 間直近に電解 100μF 程度を追加して様子を見る。※ 電源スイッチは設けない。ピン16 → MP1584EN の 12V 線は、第10章の CAN ID 特定作業が終わるまで**接続しない**（USB 給電で作業する）。12V 線を繋いだ後は、USB を挿すときは装置を OBD から抜く。

5-2. 信号系統

接続元	接続先	備考
ESP32 3V3	SN65HVD230 3V3 (VCC)	トランシーバ電源。USB 給電のみでも受信動作する（第10章で利用）
ESP32 GPIO 4	SN65HVD230 R (RXD / CRX)	CAN受信
ESP32 GPIO 5	接続しない	ファームウェア上のダミー指定のみ。配線は行わない
SN65HVD230 D (TXD / CTX)	開放（何も接続しない）	送信を物理的に無効化する。 データシートのドライバ機能表で D は「High または Open → レセツプ」。D からは約 30μA が流出する仕様で、開放でも High 側に保たれる。D ピン（多くの基板で GND ピンの隣）が GND や他のピンと接触しないよう、ヘッダを残す場合はカプトンテープで絶縁する。より確実にしたい場合はジャンパ線で 3V3 へ結んでもよい（任意）
OBD-II ピン6	SN65HVD230 CANH	CAN-C 高速バス
OBD-II ピン14	SN65HVD230 CANL	CAN-C 高速バス

GPIO 5 を配線せず、TXD を 3.3V に固定する理由

ファームウェアは TWAI を Listen Only モードで起動します（ソースコードで確認済み）。Espressif の公式ドキュメントでは、Listen Only は ACK もエラーフレームも含めドミナントビットを一切送出しない仕様で、公式エラータ（ESP32 の TWAI 関連 11 件）にも該当する項目はありません。ただし esp32.com には「Listen mode が純粋な listen ではなく ACK らしきものを出す」という未検証の報告があり、また通常モードでのビットレート不一致など設定ミスがあれば、どの CAN コントローラでもエラーフレームを出します。

本装置は送信機能を一切使わないため、トランシーバの D（TXD）には何も接続せず、ESP32 の TX ピンも配線しません。SN65HVD230 は D が High（ $V_{IH} \geq 2V$ ）または開放のとき必ずレセツプになる（データシートのドライバ機能表。D からは約 30 μA が流出する仕様で、開放でも内部的に High 側へ引かれる）ため、**ソフトウェア側に不具合や設定ミスがあっても物理的にバスを駆動できません。**

6. 電源電圧の調整

ESP32 を接続する前に必ず実施

MP1584EN の出荷時出力電圧は不定です。調整前の出力が ESP32 に加わると即座に破損します。

調整は自宅で行えます。MP1584EN の入力範囲は 4.5~28V で、出力電圧の設定は入力電圧に依存しない（フィードバックで一定に保たれる）ため、7~28V の DC 電源なら何でも使え、そこで調整した値がそのまま車両の 12~14.5V でも有効です（有力）。手持ちで使えるもの：9V 乾電池（006P）、電圧を問わず 7~28V の AC アダプタ（ルーター 12V、モニターや古いノート PC の 19~20V、LED 用 12/24V など）、USB 充電器 2 個の直列（各出力は絶縁されているので 5V+5V=10V にできる。USB ケーブルを切って赤=+5V・黒=GND を取り出す）、単 3 電池 6 本直列（アルカリ 9V / ニッケル水素 7.2V。無負荷なのでテープで束ねれば十分）。いずれも無ければ車のシガーソケットの 12V で行う（ESP32 は繋がらない）。

1. MP1584EN の IN+ / IN- に 9V 乾電池（またはその他 7~28V の DC 電源）を接続する。この時点で ESP32 とトランシーバは**接続しない**。
2. テスターを OUT+ と OUT- に当て、出力電圧を測定する。
3. 基板上の半固定抵抗（ポテンショメータ）を精密ドライバーでゆっくり回し、**5.00V $\pm$ 0.05V** に調整する。
4. 入力を切り、再投入して電圧が再現することを確認する。
5. **調整完了後、半固定抵抗にマニキュアまたは接着剤を少量塗布して固定する。**車両の振動で設定値がずれるのを防ぐため、この工程は省略しないでください。
6. 電源を切ってから、ESP32 の 5V ピンへ接続する。

7. ソフトウェア環境の構築

7-1. Arduino IDE の準備

- Arduino IDE をインストールし、ボードマネージャで **esp32 by Espressif Systems (3.x 系)** を導入する。本書は 3.3.11 でビルドを確認済み。作者の開発環境も 3.x 系。
- macOS で書き込みポートが表示されない場合は、CH340 ドライバ（Freenove の GitHub リポジトリに同梱）を導入する。

7-2. 必要ライブラリ

ライブラリ	導入方法	備考
NimBLE-Arduino	ライブラリマネージャでバージョン 1.4.3 を明示的に選択	BLE スタック。既定でインストールされる 2.x（現行 2.5.1）では arduino-RaceChrono がビルドできない（setScanResponse が廃止、onWrite の引数に変更）。1.4.3 は 1.x 系の最終版で、esp32 3.x 系に対応。
EasyLogger	ライブラリマネージャ（1.1.4 で確認）	シリアルログ出力
arduino-RaceChrono	手動インストール	ライブラリマネージャには登録されていない。下のコマンドで ~/Documents/Arduino/libraries に配置する（macOS の Arduino IDE の既定スケッチブック）。

```
cd ~/Documents/Arduino/libraries
git clone https://github.com/timurrrrr/arduino-RaceChrono.git arduino-RaceChrono
```

7-3. ソースコードの取得と config.h の作成

1. GitHub の kevenduchesneau/racechrono-diy-esp32 からソース一式をダウンロードする（本書は上流コミット c2290a9 を基準にしている）。
2. config.h.example を config.h にリネーム（またはコピー）する。このファイルが無いとビルドできません。
3. config.h の内容を次節の通りに書き換える。
4. CAN ID の特定を行う場合は、付録 A の差分を racechrono-diy-esp32.ino に当てておく（通常運用時は CAN_SNIFFER 0 のままで、コードは組み込まれない）。

7-4. config.h（全文）

```
#ifndef CONFIG_H
#define CONFIG_H

// RaceChronoアプリ上に表示されるBluetoothデバイス名
#define DEVICE_NAME "RaceChrono DIY ESP32"

// シリアルログレベル
// CAN ID特定作業中は LOG_LEVEL_DEBUG に変更する
#define LOG_LEVEL LOG_LEVEL_INFO

// CAN受信ピン：SN65HVD230 の R (RXD) へ接続
#define RX_PIN 4

// CAN送信ピン：【意図的に未接続】ドライバ起動用のダミー指定
// トランシーバの D (TXD) には何も接続しない（開放＝レセプティブ固定）
#define TX_PIN 5

// シリアル通信速度
#define SERIAL_BAUD_RATE 115200

// TWAI受信バッファ長
// 起動時に "Failed to create ring buffer." が出る場合は 64 に減らす
#define TWAI_RX_QUEUE_LENGTH 128

// BLE 5.0 拡張アドバタイズ
// 無印ESP32は BT 4.2 のため有効化不可。コメントアウトのままとする
// #define CONFIG_BT_NIMBLE_EXT_ADV 1

// 全PID共通の送信レート上限 (Hz)
#define DEFAULT_UPDATE_RATE_HZ 2

// PIDごとのレート上書き (Fiat 500系のIDは29ビット。第3章の候補表を参照)
```

```
uint8_t getUpdateRateHz(uint32_t can_id)
{
    switch (can_id)
    {
        {
            // 記入例（IDが確認できたらコメントを外す）：
            // case 0x0218A006:    // 4輪速
            //     return 20;
            default:
                return DEFAULT_UPDATE_RATE_HZ;
        }
    }

    // CAN ID 発見用スニファ（第10章）。1 で有効、通常運用時は 0
    #define CAN_SNIFFER 0
    // 最初のフレーム受信からこの秒数を「基準」とし、常に変化するバイトを覚える
    #define SNIFFER_BASELINE_SEC 30
    // スニファの表示周期（秒）
    #define SNIFFER_REPORT_SEC 1
    // 追跡するIDの最大数
    #define SNIFFER_MAX_IDS 256

    #endif // CONFIG_H
```

7-5. Arduino IDE のボード設定

リポジトリの README は ESP32-S3 を前提としているため、無印 ESP32 では以下のように読み替えます。「ESP32 Dev Module」の既定値と異なるのは Upload Speed のみです。

README の記載	無印ESP32での設定
Board	ESP32 Dev Module
USB CDC On Boot	項目なし → 無視
Upload Mode: USB-OTG CDC (TinyUSB)	項目なし → 無視（通常の UART 書き込み）
USB Mode: Hardware CDC and JTAG	項目なし → 無視
Events Run On: Core 1	Core 1（既定値。作者は必須と明記）
Arduino Runs On: Core 1	Core 1（既定値。作者は必須と明記）
Partition Scheme: Default	Default 4MB with spiffs（1.2MB APP）のまま。ビルド結果 603,600 bytes（46%）で収まる
Upload Speed	115200（921600 でも書けるが、失敗時は 115200 に落とす）

7-6. 書き込みと起動確認

1. 装置を車両（OBD）に繋いでいない状態で、USB ケーブルで PC と接続する。
2. ポートを選択し、スケッチを書き込む。
3. シリアルモニタ（115200bps）を開き、起動ログに Driver started. と BLE started. が出ることを確認する。

USB 給電と車両 12V の同時接続について

Freenove ボードは回路図が公開されておらず、USB の 5V と 5V ピンの間に逆流防止ダイオードがあるかは不明です。本書では「USB 給電と 12V 給電を同時にしない」を原則とします。書き込み・シリアル確認はすべて USB 給電のみで行い、車内でのシリアル作業（第10章）はピン16 → MP1584EN の 12V 線を繋ぐ前に済ませます（トランシーバは ESP32 の 3V3 から給電されるため、USB 給電だけで CAN 受信も動作します）。12V 線を繋いだ後に USB を使うときは、装置を OBD から抜きます。同時給電が必要な場合は、USB を外した状態でテスターのダイオードモードで USB コネクタの VBUS と 5V ピン間を測り、ダイオード（順方向 0.2～0.7V 程度、逆方向は OL）が確認できた場合に限り可とします。

8. 性能に関する期待値

- 作者の README では、ESP32-S3 で 120Hz 級（合計では約 200Hz、Google Pixel 7 で計測）に達し、シングルコアの ESP32-C3 でも同等、BLE 5.0 の拡張アダプタイズを有効にしても差が無かったと記載されています。ボトルネックは BLE リンクの転送速度です。
- したがって BT 4.2 の無印 ESP32 でも S3 と同程度が見込めます（有力）。実効レートはスマートフォン側（iOS は接続間隔の下限が長め）に左右されるため、**実測して決めます**。
- 設定方針：DEFAULT_UPDATE_RATE_HZ は 2 から始め、本当に必要なチャンネル（舵角・ブレーキ・アクセル・車速/回転）のみ getUpdateRateHz() で 20~25Hz に引き上げる。すべての PID を高レートで通そうとすると BLE 側で詰まり全体のレートが落ちます。
- 実測方法：LOG_LEVEL_DEBUG にすると 10 秒ごとに「messages received from TWAI」「messages sent to ring buffer」「messages sent to RaceChronoBle」が出ます。ring buffer 行と RaceChronoBle 行の差が広がる、または Receive ring buffer overflow が出るなら BLE が飽和しています。レートを下げてください。

9. ベンチテスト（車両接続前の必須確認）

以下をすべて満たすまで、車両には接続しないでください。CAN の受信そのものは本 BOM では机上で確認できない（送信側が無い）ため、第10章の初回接続で確認します。

9-1. 通電前（テスター）

- ☐ SN65HVD230 基板の CANH-CANL 間が 120Ω 付近でない（120Ω 除去済み。数十 kΩ 以上または OL）
- ☐ MP1584EN の出力が 5.00V ± 0.05V で安定しており、半固定抵抗が固定されている
- ☐ OBD-II ピン16 → MP1584EN IN+、ピン4/5 → IN- の極性が正しい（逆接続保護は無い）
- ☐ ESP32 の GPIO 5 からトランシーバへの配線が**存在しない**
- ☐ トランシーバの D (TXD / CTX) ピンに何も接続されておらず、GND や隣のピンと接触していない（テスターで D-GND 間が導通しない）
- ☐ 12V ラインと GND ラインの間にショートが無い
- ☐ すべての配線に熱収縮チューブまたはカプトンテープで絶縁処理がされている

9-2. USB 給電（12V 未接続）

- ☐ シリアルに起動ログが出る（Driver started. / BLE started.）
- ☐ スマートフォンで「RaceChrono DIY ESP32」が見える（iOS は設定 > Bluetooth の一覧、または BLE スキャナアプリ）
- ☐ RaceChrono Pro からデバイスを追加し、接続が確立する（シリアルに RaceChrono connected.）

9-3. 外部給電（USB を外し、9V 乾電池または 12V アダプタを OBD 側の 12V/GND 配線へ）

- ☐ 給電すると ESP32 が起動し、スマートフォンから BLE が見える・接続できる（USB は外しておく）
- ☐ MP1584EN・ESP32 に発熱・異臭が無い。テスターで ESP32 の 5V ピンが 5.0V 付近
- ☐ 給電を外すと BLE が消える

10. 車両接続と CAN ID 特定作業

10-1. 初回接続（受信確認）

1. config.h の LOG_LEVEL を LOG_LEVEL_DEBUG にして書き込んでおく（CAN_SNIFFER は 0 のまま）。
2. ピン16 → MP1584EN の 12V 線は**まだ接続しない**（USB 給電で作業する）。OBD-II ポート（運転席足元・低い位置）に接続する。ケーブルは上方向へ逃がし、結束バンドで既存ハーネスへ固定する。
3. ノート PC の USB で給電し、シリアルモニタ（115200bps）を開く。
4. イグニッションを ON にする（エンジン始動でもよい）。

5. 10 秒ごとに `N messages received from TWAI in the last 10 seconds.` が出て、N が 0 より大きければ受信成功。0 のままなら第11章へ。
6. CAN ID の特定（10-2）まで USB 給電のまま進める。終わったら装置を OBD から抜き、ピン16 → MP1584EN の 12V 線を接続して、USB なしで OBD に挿し直し、RaceChrono Pro から接続できることを確認する。

ここでフレームが流れてくれば、本プロジェクトは9割成功です。

10-2. CAN ID の特定（スニファ）

RaceChrono には未知の CAN ID を一覧化したり、全フレームを保存したりする機能はありません（プロトコル上の「allow all」はアプリが PID 編集画面を開いたときに内部的に送るコマンドで、ユーザー操作ではありません）。また上流ファームウェアの DEBUG ログは受信件数しか出しません。そこで付録 A の差分で追加するスニファを使います。

準備：付録 A の差分を当てた `.ino` で、`config.h` の `CAN_SNIFFER` を 1 にして書き込む。BLE 機能はそのまま動くので RaceChrono を同時に接続していても構いません。特定作業が終わったら 0 に戻して書き込み直します。

動作：最初のフレームを受信してから 30 秒間（`SNIFFER_BASELINE_SEC`）を「基準」として、常に変化しているバイト（カウンタ・チェックサム・センサの揺らぎ）を覚えます。その後は 1 秒ごとに、**基準期間には静止していたのに変化したバイト**を持つ ID と、新しく現れた ID だけを表示します。操作は不要で、シリアルモニタを見るだけです。

```
[sniffer] enabled, waiting for the first CAN frame
[sniffer] first frames seen, baseline for 30 s: do not touch anything
[sniffer] baseline... 57 IDs, 2913 frames/s
[sniffer] baseline done: 57 IDs, 2915 frames/s. Operate ONE control at a time.
--- t=48s 57 IDs 2910 frames/s ---
    0x0218A006  100Hz  ..  .. *3A *F1  00  00  00  00
NEW 0x0A18A002   5Hz  AA  BB
（表示例。ID と値は架空）
```

1 行の意味：先頭の NEW は初出の ID。ID は 29 ビットなら 8 桁、11 ビットなら 3 桁の 16 進で表示。Hz はその ID の受信レート。データ部は *XX が「今変化した（基準では静止）」、.. が「常に変化（ノイズ）」、XX が静止中の値です。

手順（停車・エンジン始動状態、12V 線は未接続・USB 給電）：

- 既知候補の確認：**第3章の候補表の ID（`0x0218A006` 車輪速、`0x0210A006` 車速、`0x0618A001` 回転数など）がスニファの ID 一覧に現れるか、ベースライン中の表示（ID 数と frames/s）と合わせて確認する。現れれば RaceChrono に候補の式を入れて値が追従するか試す。
- シリアルモニタを開き、`[sniffer] first frames seen, baseline for 30 s` が出たら何も触らずに待つ。`[sniffer] baseline done` で準備完了。
- 舵角：**ステアリングをロック to ロックでゆっくり 1 往復させる。この間だけ * 付きで出続ける ID が候補。ロック to ロックで値が単調に増減する 1~2 バイト（16 ビット符号付きが多い）を探す。
- ブレーキ：**ペダルをゆっくり全ストロークさせる。圧力は連続値、ブレーキランプ SW は 1 ビットの変化として現れる。
- アクセル：**ゆっくり踏み込む（周囲の安全と騒音に配慮）。回転数と開度が同時に動くので、開度だけを見たい場合はエンジン停止・イグニッション ON でも試す（開度が出るかは要確認）。
- 各候補 ID について、RaceChrono の CAN-Bus チャンネル編集で PID と計算式（`bytesToUIntLe` 等）を入力し、実車操作に値が追従するか確認する。

※ 表示が多すぎる場合は `SNIFFER_BASELINE_SEC` を延ばす（ノイズ学習が増える）。追跡できる ID は最大 256（`SNIFFER_MAX_IDS`）。特定した ID は `getUpdateRateHz()` に追記して優先度を上げる。

補足：診断フレームとの見分け方

診断（KWP/UDS）のフレームは 0x18DAxxF1 / 0x18DBxxF1（要求）と 0x18DAF1xx（応答）の形で、診断機を繋いだときだけ流れます。常時放送のフレームは 0x0XXXA0YY の形の 29 ビット ID です。ファームウェアは全 ID 受け入れ設定なので両方受信します。RaceChrono の PID 欄には 29 ビット ID をそのまま入力します（表記形式は実機で確認）。

代替手段：OBDLink EX（MultiECUScan 用）で先に ID を確認する

MultiECUScan 自体は診断要求（0x18DA10F1 など）を投げて応答を読む方式で、常時放送フレームの ID は表示できません。ただし同じ OBDLink EX は STN コマンドでそのまま CAN スニファになるため、DIY 機を組む前に第 3 章の候補 ID を実車で確認できます。Mac のターミナルで `ls /dev/tty.usbserial*` でポート名を確認し、次で接続します。

```
screen /dev/tty.usbserial-XXXX 115200
```

接続後、1 行ずつ送ります（Enter で確定）。

```
ATZ
ATE0
ATH1
ATCAF0
ATS1
STP 32
STMA
```

STP 32 は ISO 11898 生 CAN・500kbps、STMA で全フレームが「ID データ」の形で流れ続けます（STN リファレンスによれば CAN プリセットは 11 ビット / 29 ビットの両方を受信）。止めるときは何かキーを押す。screen の終了は Ctrl-A、K、y。

- 500kbps のバスは毎秒数千フレームなので 115200 では取りこぼして BUFFER FULL が出る。眺めるだけなら実害なし。候補 ID だけ見るときは STFPA 0218A006, 1FFFFFFF のようにパスフィルタを追加してから STM。全量が必要なら STBR 2000000 で 2Mbps に上げる（screen は 2Mbps 非対応のため pyserial 等を使う）。
- MultiECUScan と同時には使えない。舵角などの基準値は MultiECUScan の Live Data で別途読み、静止状態（直進・ロック位置など）で照合する。
- 診断機の要求 / 応答（0x18DA...）は自分の通信なので無視する。

10-3. RaceChrono Pro 側の設定

- 公式チュートリアル経路は「Settings → Add other device → Add other device → RaceChrono DIY」（日本語 UI では「設定 → その他のデバイスを追加 → …」）。BLE スキャンの一覧から **RaceChrono DIY ESP32** を選ぶ。
- デバイス追加後、車両プロファイルに「CAN-Bus チャンネル」の項目が現れる。特定した CAN ID と計算式を 1 チャンネルずつ登録する。

※ DIY BLE デバイスの追加と CAN-Bus チャンネル機能には RaceChrono Pro（有料版）が必要です。画面名や並びはアプリのバージョンで変わるため、実機で確認したうえで本書を更新してください（Rev.1.0 の「デバイス種別を CAN-Bus reader として設定する」は公式資料に該当する記述が無いため削除）。

11. トラブルシューティング

症状	確認事項
ビルドエラー： <code>'class NimBLEAdvertising' has no member named 'setScanResponse'</code>	NimBLE-Arduino が 2.x になっている。ライブラリマネージャで 1.4.3 を選んで入れ直す（第7章）。
ビルドエラー：RX_PIN 等が未定義	<code>config.h.example</code> を <code>config.h</code> にリネームしたか確認。マクロ名は <code>CAN_RX_PIN</code> ではなく <code>RX_PIN</code> 。
ビルドエラー：RaceChrono.h が見つからない	arduino-RaceChrono ライブラリを <code>libraries</code> フォルダへ手動配置したか確認（7-2）。
ビルドエラー： <code>driver/twai.h</code> や <code>esp_mac.h</code> が見つからない	ESP32 ボードパッケージが古い、または壊れている。esp32 3.x 系を入れ直す。
起動時に <code>Failed to create ring buffer.</code>	RAM 不足。 <code>.ino</code> 内の <code>xRingbufferCreate(65536, ...)</code> を 16384 に、 <code>TWAI_RX_QUEUE_LENGTH</code> を 64 に減らす。
起動時に <code>Driver failed to start.</code>	<code>RX_PIN</code> / <code>TX_PIN</code> の GPIO 番号を確認。 <code>TX_PIN</code> は未配線でも有効な GPIO 番号を指定する必要がある。
BLE で見えるが TWAI の受信数が 0 のまま / スニファが <code>no CAN frames yet</code>	イグニッション ON か。GPIO 4 とトランシーバ RXD の結線、トランシーバの 3V3 給電、CANH/CANL の入れ違い（ピン 6=H、ピン 14=L）、終端抵抗除去時のパターン断線を確認。
受信数は増えるが RaceChrono に値が出ない	PID・計算式の誤り。DEBUG ログの <code>Allow PID ...</code> でアプリからの要求が届いているか確認。
フレームが一切受信できない（上記を確認済み）	ビットレートを確認。500kbps 以外の可能性がある場合は、車両から切り離した上で <code>.ino</code> の <code>TWAI_TIMING_CONFIG_500KBITS()</code> を変更する。
データレートが遅い・途切れる	BLE の帯域上限。第8章の指標で飽和を確認し、 <code>DEFAULT_UPDATE_RATE_HZ</code> を下げ、必要な PID のみ個別に引き上げる。
ESP32 が起動しない・発熱する	直ちに電源を切る。MP1584EN の出力電圧を再測定する。
放置後にバッテリーが上がる	装置を挿したまま放置した。OBD-II ピン 16 は常時通電なので、使わないときは抜く。

12. 運用上の注意

- **使わないときは装置を OBD ポートから抜く。** 本装置は挿している間、12V 側で数十 mA を消費し続けます（目安として 1 日あたり 1Ah 前後）。挿したまま放置すると 1~2 週間でバッテリーに影響が出ると見込んでください。
- 夏季の車内は高温になります。長期駐車時は本体を取り外すことを推奨します。
- OBD ポート周辺は乗降時に膝が当たる位置です。本体とケーブルが運転操作を妨げないように固定してください。
- 走行中に操作が必要な状態にしないでください。設定変更は必ず停車時に行ってください。
- 本装置は受信専用ですが、車両の電気系統に接続する改造行為です。実施は自己責任で行ってください。
- 将来の改善案：CAN の無音（車両スリープ）を検出して deep sleep に入り、GPIO 4 の信号で復帰する処理を入れると、挿しっぱなしでも消費を抑えられます。

参考リンク

- ファームウェア：github.com/kevenduchesneau/racechrono-diy-esp32
- ライブラリ：github.com/timurrrr/arduino-RaceChrono

- BLE DIY プロトコル仕様：github.com/aollin/racechrono-ble-diy-device
- CAN ID 事例集（他車種の参考）：github.com/timurrrr/RaceChronoDiyBleDevice
- RaceChrono DIY チュートリアル：racechrono.com/article/2572 / DIY フォーラム：racechrono.com/forum/categories/diy-builds
- NimBLE-Arduino 1.x→2.x 移行ガイド：github.com/h2zero/NimBLE-Arduino/blob/master/docs/1.x_to2.x_migration_guide.md
- ESP-IDF TWAI ドキュメント／ESP32 エラッタ：docs.espressif.com
- Freenove ESP32-WROOM ボード（ピン配置・CH340 ドライバ）：github.com/Freenove/Freenove_ESP32_WROOM_Board
- Fiat/Alfa の OBD ピン割当（C-CAN 6/14、B-CAN 1/9）：alfaobd.com/androidhelp.html
- STN11xx/21xx Family Reference and Programming Manual（OBDLink の STM/STMA/STFPA/STBR/STP）：scantool.net
- Fiat 500（2010）CAN 解析ノート：github.com/P1kachu/talking-with-cars/blob/master/notes/fiat-500.txt
- Fiat 500L CAN 復号（FiatMon）：github.com/VeryBusyBee/FiatMon
- Racelogic Vehicle CAN Database「Fiat 500 2007-」：racelogic.co.uk（PDF）／vboxmotorsport.co.uk（.REF）
- AiM InfoTech「Fiat 500 Abarth (from 2008 onward)」：aim-sportline.com/download/ecu/stock/fiat/FIAT500Abarth_106_eng.pdf

付録 A. CAN ID スニファ（.ino への差分）

上流コミット c2290a9 の `racechrono-diy-esp32.ino` に対する追加です。変更は 3 箇所、`CAN_SNIFFER` が 0 のときはコードが一切組み込まれません。ESP32 Dev Module 向けにビルド確認済み（605,240 bytes、46%）。ロジックは PC 上で合成フレームを流して検証しましたが、実車での動作は未検証です。

A-1. config.h

7-4 の全文に含まれている `CAN_SNIFFER` / `SNIFFER_BASELINE_SEC` / `SNIFFER_REPORT_SEC` / `SNIFFER_MAX_IDS` の 4 つの定義を使います。

A-2. racechrono-diy-esp32.ino

(1) `void taskGetTwaiMessages(void *)` の直前に、次のブロックを丸ごと追加する

```
#if CAN_SNIFFER
// -----
// CAN ID sniffer (debug aid for finding CAN IDs, see build manual chapter 10)
// Learns which bytes are always changing during a baseline window, then
// reports only the IDs / bytes that change afterwards. Non-interactive.
// -----
struct SnifferEntry
{
    uint32_t id;
    uint8_t extd;
    uint8_t dlc;
    uint8_t data[8];
    uint8_t changedMask; // bytes changed since the last report
    uint8_t noisyMask;   // bytes that changed during the baseline window
    uint32_t count;      // frames since the last report
    bool isNew;          // first seen since the last report
};

static SnifferEntry snifferTable[SNIFFER_MAX_IDS];
static SnifferEntry snifferSnapshot[SNIFFER_MAX_IDS];
static int snifferCount = 0;
static portMUX_TYPE snifferMux = portMUX_INITIALIZER_UNLOCKED;

// Called from taskGetTwaiMessages() for every received frame (fast path)
static void snifferRecord(const twai_message_t *m)
{
    portENTER_CRITICAL(&snifferMux);
    int i = 0;
    for (; i < snifferCount; i++)
    {
        if (snifferTable[i].id == m->identifier && snifferTable[i].extd == m->extd)
        {

```

```

        break;
    }
}
if (i == snifferCount)
{
    if (snifferCount < SNIFFER_MAX_IDS)
    {
        SnifferEntry *e = &snifferTable[snifferCount++];
        memset(e, 0, sizeof(*e));
        e->id = m->identifier;
        e->extd = m->extd;
        e->dlc = m->data_length_code;
        memcpy(e->data, m->data, sizeof(e->data));
        e->isNew = true;
        e->count = 1;
    }
    portEXIT_CRITICAL(&snifferMux);
    return;
}
SnifferEntry *e = &snifferTable[i];
e->count++;
e->dlc = m->data_length_code;
int n = (m->data_length_code < 8) ? m->data_length_code : 8;
for (int b = 0; b < n; b++)
{
    if (e->data[b] != m->data[b])
    {
        e->changedMask |= (uint8_t)(1u << b);
        e->data[b] = m->data[b];
    }
}
portEXIT_CRITICAL(&snifferMux);
}

// One report line: "NEW 0x0218A006 100Hz 12 *34 .. 56" (29-bit IDs print 8 digits)
// *XX = changed now (was static in baseline) .. = always changing
static void snifferPrintEntry(const SnifferEntry *e, uint32_t periodMs)
{
    uint32_t hz = (periodMs > 0) ? (e->count * 1000UL) / periodMs : 0;
    if (e->extd)
    {
        Serial.printf("%s 0x%08X %4luHz ", e->isNew ? "NEW " : " ",
            (unsigned int)e->id, (unsigned long)hz);
    }
    else
    {
        Serial.printf("%s 0x%03X %4luHz ", e->isNew ? "NEW " : " ",
            (unsigned int)e->id, (unsigned long)hz);
    }
    int n = (e->dlc < 8) ? e->dlc : 8;
    for (int b = 0; b < n; b++)
    {
        uint8_t bit = (uint8_t)(1u << b);
        if ((e->changedMask & bit) && !(e->noisyMask & bit))
        {
            Serial.printf(" *%02X", e->data[b]);
        }
        else if (e->noisyMask & bit)
        {
            Serial.printf(" ..");
        }
        else
        {
            Serial.printf(" %02X", e->data[b]);
        }
    }
    Serial.println();
}

// Runs on the main core at low priority. Reports every SNIFFER_REPORT_SEC.
static void taskSniffer(void *)
{
    const uint32_t periodMs = SNIFFER_REPORT_SEC * 1000UL;

```

```

uint32_t baselineStart = 0;
bool baselineDone = false;
Serial.println("[sniffer] enabled, waiting for the first CAN frame");

for (;;)
{
    vTaskDelay(pdMS_TO_TICKS(periodMs));

    int n;
    uint32_t frames = 0;
    portENTER_CRITICAL(&snifferMux);
    n = snifferCount;
    memcpy(snifferSnapshot, snifferTable, sizeof(SnifferEntry) * n);
    for (int i = 0; i < n; i++)
    {
        SnifferEntry *e = &snifferTable[i];
        frames += e->count;
        if (!baselineDone)
        {
            e->noisyMask |= e->changedMask; // still learning
        }
        e->changedMask = 0;
        e->count = 0;
        e->isNew = false;
    }
    portEXIT_CRITICAL(&snifferMux);

    unsigned long fps = (unsigned long)(frames * 1000UL / periodMs);
    if (n == 0)
    {
        Serial.println("[sniffer] no CAN frames yet");
        continue;
    }
    if (baselineStart == 0)
    {
        baselineStart = millis();
        Serial.printf("[sniffer] first frames seen, baseline for %d s: "
            "do not touch anything\n", (int)SNIFFER_BASELINE_SEC);
    }
    if (!baselineDone)
    {
        if ((millis() - baselineStart) >= SNIFFER_BASELINE_SEC * 1000UL)
        {
            baselineDone = true;
            Serial.printf("[sniffer] baseline done: %d IDs, %lu frames/s. "
                "Operate ONE control at a time.\n", n, fps);
        }
        else
        {
            Serial.printf("[sniffer] baseline... %d IDs, %lu frames/s\n", n, fps);
        }
        continue;
    }

    int shown = 0;
    for (int i = 0; i < n; i++)
    {
        const SnifferEntry *e = &snifferSnapshot[i];
        bool interesting = e->isNew || ((e->changedMask & ~e->noisyMask) != 0);
        if (interesting)
        {
            if (shown == 0)
            {
                Serial.printf("--- t=%lus %d IDs %lu frames/s ---\n",
                    (unsigned long)(millis() / 1000UL), n, fps);
            }
            snifferPrintEntry(e, periodMs);
            shown++;
        }
    }
}

```

```
}  
#endif // CAN_SNIFFER
```

(2) `taskGetTwaiMessages()` 内、`msgCountRx++;` の直後に 3 行追加する

```
#if CAN_SNIFFER  
    snifferRecord(&message);  
#endif
```

(3) `setup()` 内、`xTaskCreatePinnedToCore(taskSendBLEMessages, ...)` の直後に追加する

```
#if CAN_SNIFFER  
    delay(1250);  
    xTaskCreatePinnedToCore(taskSniffer, "taskSniffer", 4096, NULL, 1, NULL, mainCore);  
#endif
```

※ 上流の `.ino` に対する unified diff (202 行) が必要な場合は別途用意できます。

改訂履歴

版	日付	内容
Rev. 1.0	2026-09-04	初版
Rev. 1.1	2026-09-05	7-2：NimBLE-Arduino を 1.4.3 に固定（2.x では arduino-RaceChrono がビルド不能。実際のエラーを 11 章に記載）。arduino-RaceChrono の導入コマンドを追記。 7-1：「esp_mac.h のため 2.x 系ではビルド不可」を削除（esp_mac.h は 2.x 系にも存在）。3.3.11 でのビルド確認結果を追記。 7-6／9 章／10-1：USB 給電と車両 12V の扱いを整理。書き込み・シリアル確認は「電源スイッチ OFF・USB 給電のみ」に統一。ベンチテストを通電前／USB／12V の 3 段階に再構成。 10-2：RaceChrono 側で全 PID をログする手順を削除し、付録 A のスニファによる手順に置換。 10-3：公式チュートリアル経路に修正。裏付けの無い「デバイス種別の設定」を削除し、実機確認を明記。 5 章コラム：「既知のエラッタ」を公式資料の記述に合わせて修正（設計は変更なし）。 8 章：無印 ESP32 の性能期待値を「S3 と同程度・実測で決める」に修正し、飽和の判定方法を追記。 4-1／9 章：終端抵抗除去の判定を「OL」から「120Ω 付近でない」に修正。4-3：40 ピンに修正。 3 章：2 台構成の未確認事項（B-CAN の物理層、複数 DIY デバイス）を追記。 12 章：バッテリー消費の目安を修正。部品表は変更なし。
Rev. 1.2	2026-09-05	部品表からヒューズ・逆接続保護ダイオード（1N5819）・TVS（SMBJ26A）を削除。5-1 の電源系統と 9-1 のチェックリストを追従。それ以外の章は変更なし。
Rev. 1.3	2026-09-05	10kΩ 抵抗を削除し、トランシーバの D（TXD）は 3V3 へ直結に変更（データシート上 D は High または開放でレセツプ。抵抗は不要）。5-2・5 章コラム・9-1・config.h のコメントを追従。
Rev. 1.4	2026-09-05	D（TXD）を「開放（無接続）」に変更。配線工程を減らして作業を確実にするため。D ピンの絶縁と D-GND 非導通のチェックを追加。3V3 へのジャンパ線は任意扱い。
Rev. 1.5	2026-09-05	3 章：Fiat 500（312）系の公開情報から既知 CAN ID 候補表を追加（車輪速・車速・回転数・ブレーキ・状態ビット）。放送フレームは 29 ビット ID である旨に修正（旧版の「11 ビット」は誤り）。AiM／Racelogic の資料により舵角は CAN-C 側にある見込みと記載し、2 台構成は予備案に格下げ。10-2 に「既知候補の確認」を追加、診断フレームの補足を修正。付録 A：スニファの ID 表示を 29 ビットは 8 桁に変更。
Rev. 1.6	2026-09-05	電源スイッチと外付け電解コンデンサ（100μF×2）を部品表と結線から削除（スイッチは使わないときに OBD から抜けば足りる。コンデンサは DC-DC モジュール搭載分で代替、リセットが出る場合のみ 5V 側に追加）。USB と 12V の同時給電を避ける手順を「CAN ID 特定が終わるまで 12V 線を繋がない」に変更し、7-6・9 章・10-1・12 章を追従。6 章：出力調整は 7～28V の任意の DC 電源（9V 乾電池など）で自宅のできる旨と電源の選択肢を追記。10-2：OBDLink EX を STN コマンドでスニファとして使う代替手段を追加。